

FTTRS

Field Travel Tracking & Reimbursement System

Developer Completion Guide

Backend Audit Prompts • App Improvements • Web–App Sync Plan
Based on actual screenshots of the working app (April 2026)

What we see Dashboard, Tracking map, Trip Details, Claims, Profile	What's missing Start/Pause/Stop buttons, Claim generation from trip	Goal Complete app → hand off to web for manager view
--	---	--

1. What You've Already Built — Screen-by-Screen Analysis

Based on the 8 screenshots you shared, here is exactly what is working and what needs to be completed. This gives the whole team a shared understanding of the current state.

Screen	Status	What's Working	What's Missing / Broken
Dashboard	☒ Working	Online badge, trip status card (Completed/0.00km/GPS Points), Start Trip button, mini map, pending claims, weekly compliance %	Distance shows 0.00 km even after a completed trip — distance calculation bug. 'Outside assigned geofence' warning always showing — geofence not configured.
Tracking	☒ Working	Google Maps live view, employee name label, checkpoint count, completed badge, layer toggle, recenter button	Checkpoints: 0 — checkpoint creation not implemented. Trip status shows 'completed' but map state is stale — needs real-time Socket.IO sync.
Trip Details (Map)	☒ Working	Route map embedded, COMPLETED badge, date, route length overlay with trip icon	Route length 0.00 km — polyline not being drawn / distance not computed from location points. Map centered on Seattle (hardcoded?) instead of actual trip location.
Trip Details (Stats)	☒ Working	4 stat cards: Total Distance, Duration, Avg Speed, Elevation Gain. Trip Timeline with timestamps and named stops.	Total Distance 0.00 km — same distance bug. Elevation gain (184m) seems hardcoded. Avg Speed 4.5 km/h seems wrong for a completed trip with 0 km.
Trip Details (Timeline)	☒ Working	Visual timeline with departure/arrival/end nodes, timestamps, named locations. '3 Claims Filed' row, '12 Photos' row.	Claims Filed and Photos appear to be hardcoded demo data — not pulling from real backend. Share button present but not functional.
Claims	☒ Working	Expense Center header, pending approval amount card, + New Claim button, Recent Claims section	Pending amount INR 0.00 — no claims exist yet. 'New Claim' likely not linked to a specific trip. Recent Claims section is empty.
Profile	☒ Working	Avatar with initials, role label, edit button, Employee Profile (UUID visible), Security (2FA enabled), Alerts, Region, Log out button	Region shows 'Unknown' — needs to read from backend org_settings or device. UUID exposed on screen — should show employee code or name instead.
Second Dashboard view	☒ Working	Map at top, 4 stat cards: Trip Status, Today Distance, Pending Claims, Weekly Compliance	Today Distance 0.00 km — same bug. INR 0 pending claims. 0% compliance — compliance logic not implemented.

● The #1 Critical Bug to Fix First

Total Distance is 0.00 km on ALL screens. This is the core of the whole system.
 Root cause: Either (a) GPS location points are not being saved to the backend during a trip,
 OR (b) the distance calculation function is not running when the trip ends,
 OR (c) the distance is calculated but not written back to the trips.total_distance_km column.
 Fix this one bug first — everything else (claims amount, compliance %) depends on it.

2. Backend Audit Prompts — Copy & Paste to Your AI Assistant

Use these prompts exactly as written. Paste them one at a time into Claude / ChatGPT / Cursor. Each prompt is self-contained and targets a specific backend concern you need to verify or fix.

PROMPT 1 — Distance Calculation Audit (Fix the 0.00 km bug)

Copy this entire prompt:

I have a Node.js + Express + PostgreSQL backend for a GPS trip tracking app. The app records location points during a trip (table: locations with columns: trip_id, latitude, longitude, recorded_at). When the trip ends (POST /trips/:id/end), it should calculate total distance using the Haversine formula across all location points and write it to trips.total_distance_km.

Current problem: total_distance_km is always 0.00 after trip end.

Please audit the following and fix all issues:

1. Show me the correct SQL query to fetch all location points for a trip ordered by recorded_at.
2. Write a TypeScript haversine function that takes an array of {latitude, longitude} objects and returns total km (filtering out noise < 5m).
3. Show me how to call this function inside the trip end handler and UPDATE the trips table.
4. Write a one-time SQL migration to recalculate total_distance_km for all existing completed trips using their stored location points.

PROMPT 2 — Trip Lifecycle Endpoints Audit

Copy this entire prompt:

Audit my trip lifecycle backend for a React Native field tracking app. The app has these buttons: Start Trip, Pause, Resume, End Trip.

I need you to verify and write these 4 Express endpoints in TypeScript:

1. POST /api/trips/start
 - Validate: employee has no currently active or paused trip (prevent duplicate active trips)
 - Create trip record: status="active", start_time=NOW(), employee_id from JWT
 - Return: { trip_id, status, start_time }
2. PATCH /api/trips/:id/pause
 - Validate: trip belongs to authenticated employee, status must be "active"
 - Update: status="paused", record paused_at timestamp in a trip_pauses table
 - Return: { trip_id, status, paused_at }
3. PATCH /api/trips/:id/resume
 - Validate: status must be "paused"
 - Update: status="active", calculate and add pause_duration_secs
 - Return: { trip_id, status, resumed_at }

4. POST /api/trips/:id/end
 - Validate: status must be "active" or "paused"
 - Fetch all location points, calculate total_distance_km using Haversine
 - Update trip: status="completed", end_time=NOW(), total_distance_km
 - Auto-create a claim record: distance_km, rate_per_km from org_settings, calculated_amount
 - Emit Socket.IO event: trip:ended to manager room
 - Return: { trip_id, status, total_distance_km, claim_id, claim_amount }

Use PostgreSQL with parameterized queries. Include proper error handling.

PROMPT 3 — Real-Time Location Ingestion Audit

Copy this entire prompt:

Audit my Socket.IO location ingestion pipeline for a GPS tracking app.

Setup: Node.js + Socket.IO v4 + PostgreSQL + Redis (Upstash).

Mobile app emits: socket.emit("location:update", payload) every 15 seconds.

Write the complete Socket.IO handler for the /tracking namespace that does ALL of the following:

1. Authentication middleware: validate JWT from socket handshake auth.token, attach user to socket.data
2. On connection: join employee to room "employee:{userId}", log connection
3. On "location:update" event:
 - a. Validate payload with Zod schema: { trip_id, latitude, longitude, accuracy_m, speed_kmh, is_mock, recorded_at }
 - b. Verify trip_id belongs to authenticated employee and is in "active" status
 - c. Detect mock/spoofed GPS: if is_mock=true, add "MOCK_LOCATION" to trips.fraud_flags JSONB array
 - d. Add to in-memory batch buffer (flush to DB every 5 seconds via setInterval)
 - e. Update Redis key "live:{employee_id}" with { lat, lng, speed_kmh, trip_id } TTL 120s
 - f. Broadcast to all manager rooms that have this employee assigned: emit "location:broadcast" with { employee_id, lat, lng, speed_kmh, trip_id }
4. On disconnect: log last seen, mark employee offline in Redis

Include the batch INSERT function for locations table (bulk insert for performance).

Include the Redis broadcast logic to find which manager rooms to broadcast to.

PROMPT 4 — Claims API Audit

Copy this entire prompt:

Audit and complete my claims API for a field trip reimbursement system.

Current situation: Claims are auto-created when a trip ends, but the employee also needs to be able to manually generate a claim from a completed trip.

Write these endpoints:

1. GET /api/claims — Employee sees their own claims (role: employee)
Response: array of { claim_id, trip_id, distance_km, calculated_amount, status, created_at, reviewed_at, rejection_reason }
2. POST /api/claims/generate/:trip_id — Employee manually generates claim from completed trip
 - Validate: trip belongs to employee, status="completed", no existing claim for this trip
 - Read rate_per_km from org_settings table
 - Create claim with status="pending"
 - Send Expo push notification to manager: "New claim submitted by {employee_name}"
 - Response: { claim_id, amount, trip_id }
3. PATCH /api/claims/:id/approve — Manager approves (role: manager)
 - Validate: claim is in pending status, manager has this employee assigned
 - Update: status="approved", reviewed_by, reviewed_at, approved_amount (default = calculated_amount)
 - INSERT into approval_logs
 - Send Expo push notification to employee: "Your claim of INR {amount} was approved"
4. PATCH /api/claims/:id/reject — Manager rejects (role: manager)
 - Validate: rejection_reason is required, minimum 10 characters
 - Update: status="rejected", rejection_reason, reviewed_by, reviewed_at
 - INSERT into approval_logs
 - Send Expo push notification to employee: "Your claim was rejected: {reason}"
5. GET /api/claims (manager) — Manager sees all claims from their assigned employees
Filter params: status, employee_id, date_from, date_to
Response includes employee name, trip date, distance, amount

Use TypeScript, Express, parameterized pg queries.

PROMPT 5 — Database & Data Integrity Check

Copy this entire prompt:

Run a data integrity audit on my PostgreSQL database for a GPS tracking app. Execute these queries and report any issues:

- ```
-- 1. Check for completed trips with 0 distance but existing location points
SELECT t.id, t.status, t.total_distance_km, COUNT(l.id) as location_count
FROM trips t LEFT JOIN locations l ON l.trip_id = t.id
WHERE t.status = 'completed' AND t.total_distance_km = 0
GROUP BY t.id HAVING COUNT(l.id) > 0;

-- 2. Check for trips with no associated claim
SELECT t.id, t.employee_id, t.total_distance_km, t.status
FROM trips t LEFT JOIN claims c ON c.trip_id = t.id
WHERE t.status = 'completed' AND c.id IS NULL;

-- 3. Check for multiple active trips per employee (should never happen)
```

```
SELECT employee_id, COUNT(*) as active_count
FROM trips WHERE status IN ('active', 'paused')
GROUP BY employee_id HAVING COUNT(*) > 1;
```

```
-- 4. Check locations without a valid trip reference
SELECT COUNT(*) FROM locations l
LEFT JOIN trips t ON t.id = l.trip_id WHERE t.id IS NULL;
```

```
-- 5. Verify indexes exist
SELECT indexname, tablename FROM pg_indexes
WHERE tablename IN ('trips', 'locations', 'claims');
```

For each issue found, provide the SQL fix. Also show me how to set up a DB trigger that automatically recalculates total\_distance\_km when a trip status changes to "completed".

## PROMPT 6 — Push Notifications Setup

### Copy this entire prompt:

Set up Expo Push Notifications in my Node.js backend for a React Native app.

Requirements:

1. Install and configure expo-server-sdk in the backend
2. Write a reusable sendPushNotification(expoPushToken, title, body, data) function
3. Store Expo push tokens: add expo\_push\_token column to users table, update it via PATCH /api/users/push-token endpoint (called on app startup)
4. Send notifications for these events:
  - When a claim is approved: notify employee
  - When a claim is rejected: notify employee with reason
  - When a new claim is submitted: notify the employee's assigned manager
  - When a trip ends: notify the manager
5. Handle notification errors gracefully (invalid token = remove from DB, rate limit = retry queue)

Also show me the React Native side:

- How to request push permission with expo-notifications
- How to get the Expo push token
- How to send it to the backend on app startup
- How to handle a notification tap (navigate to the relevant claim screen)

### 3. App Improvement Plan — Based on Your Screenshots

#### 3.1 Critical Fixes (Do Before Anything Else)

| # | Screen       | Issue Seen                                         | Fix Required                                                                                        |
|---|--------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1 | Dashboard    | Distance: 0.00 km despite completed trip           | Fix Haversine calculation in POST /trips/:id/end — see Prompt 1                                     |
| 2 | Dashboard    | 'Outside assigned geofence' shown always           | Either implement real geofence per employee OR hide this warning until geofences are configured     |
| 3 | Trip Details | Route length 0.00 km, map centered on Seattle      | Fix distance bug + pass actual start_lat/start_lng to map; remove hardcoded Seattle coordinates     |
| 4 | Trip Details | 3 Claims Filed & 12 Photos are hardcoded demo data | Replace with real API calls: GET /trips/:id/claims and GET /trips/:id/photos                        |
| 5 | Profile      | UUID shown under Employee Profile                  | Show employee_code or 'EMP-XXXX' format. UUID is an internal implementation detail, not user-facing |
| 6 | Profile      | Region: Unknown                                    | Pull region from org_settings.region or add a user location preference setting                      |

#### 3.2 Missing Core Feature: Trip Control Buttons

The most important missing piece is the Start / Pause / Resume / Stop buttons with claim generation. Here is the exact UI flow and backend connections needed:



##### Required Trip Control Flow

- STATE 1 — No active trip → Show large 'Start Trip' button (already visible on Dashboard ☒)  
→ On tap: call POST /api/trips/start → get trip\_id → navigate to Active Trip Screen
- STATE 2 — Trip active → Show: [ || Pause] [■ End Trip] + live timer, distance counter  
→ Pause tap: call PATCH /api/trips/:id/pause → update button to [▶ Resume] [■ End Trip]  
→ Resume tap: call PATCH /api/trips/:id/resume → back to active state  
→ End Trip tap: show confirmation modal → call POST /api/trips/:id/end
- STATE 3 — Trip ended → Show 'Generate Claim' button on the Trip Details screen  
→ Claim button tap: call POST /api/claims/generate/:trip\_id  
→ On success: show claim amount (e.g. INR 99.20) + 'Claim Submitted' banner  
→ Navigate to Claims screen showing new pending claim

#### Active Trip Screen — Required Components

| Component     | What It Shows                              | Data Source                                                         |
|---------------|--------------------------------------------|---------------------------------------------------------------------|
| Live Timer    | HH:MM:SS counting up from trip start       | trip.start_time from local state — do NOT call API every second     |
| Live Distance | X.XX km — updates every GPS point received | Cumulative Haversine calc from location points array in local state |
| Current Speed | X.X km/h                                   | From expo-location coords.speed (m/s × 3.6)                         |

| Component         | What It Shows                         | Data Source                                                |
|-------------------|---------------------------------------|------------------------------------------------------------|
| GPS Points Count  | N points recorded                     | Counter in local state, incremented on each Socket.IO emit |
| Pause button      | Changes label/icon when paused        | Local state: isPaused boolean                              |
| End Trip button   | Red, confirmation modal before action | Modal with trip summary before final submit                |
| Connection status | Online / Offline badge                | NetInfo from @react-native-community/netinfo               |

### Trip Details Screen — Add Generate Claim Button

```
// In TripDetails screen — show Generate Claim button only if:
// 1. trip.status === 'completed'
// 2. trip.claim_id is null (no claim exists yet)

{trip.status === 'completed' && !trip.claim_id && (
 <TouchableOpacity onPress={handleGenerateClaim} style={styles.claimBtn}>
 <Text>Generate Reimbursement Claim</Text>
 <Text style={styles.amount}>
 Est. INR {(trip.total_distance_km * ratePerKm).toFixed(2)}
 </Text>
 </TouchableOpacity>
)}
```

## 3.3 UX Improvements (High Impact, Low Effort)

| Improvement                                   | Screen                  | Why                                          | Implementation                                                                      |
|-----------------------------------------------|-------------------------|----------------------------------------------|-------------------------------------------------------------------------------------|
| Replace UUID with friendly ID on Profile      | Profile                 | UUID is confusing for employees              | Show first 8 chars or use employee_code: EMP-E87E14CC                               |
| Fix 'Unknown' location with reverse geocoding | Trip Details, Dashboard | Named locations build trust with employees   | Use expo-location reverseGeocodeAsync() on trip start/end coords — free, no API key |
| Add 'No trips yet' empty state                | Trips tab               | App looks broken when list is empty          | Show illustration + 'Start your first trip' CTA button                              |
| Show claim amount on claim card               | Claims screen           | INR 0.00 pending is confusing                | Pull and display calculated_amount even before approval                             |
| Add offline indicator banner                  | All screens             | Employee may lose signal mid-trip            | Use NetInfo — show red 'Offline — GPS being saved locally' banner                   |
| Confirmation modal before End Trip            | Active trip             | Accidental trip endings lose data            | Modal showing: distance, duration, estimated claim amount                           |
| Auto-center map on employee's actual location | Tracking, Trip Details  | Map shows Seattle instead of actual location | Pass start_lat/start_lng to map initialRegion on mount                              |
| Format duration as readable                   | Trip stats              | 02:45 is ambiguous (hours? minutes?)         | Show '2h 45m' or '45 min' with label                                                |

## 3.4 New Screens to Add (Post-Fix)

| Screen                        | Purpose                                              | Key Components                                                                                         |
|-------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Active Trip Screen            | Dedicated fullscreen view during a trip              | Live timer, distance, speed, Pause/Resume/End buttons, mini map, offline queue status                  |
| Claim Detail Screen           | View a single claim's full info                      | Trip summary, amount breakdown (distance × rate), status badge, rejection reason if rejected, timeline |
| Claim Generation Confirmation | Before submitting a claim                            | Trip info, calculated amount, optional notes field, Submit button                                      |
| Trip Share Screen             | Share trip summary (the share icon is already there) | Shareable card: date, distance, duration, route map screenshot                                         |

## 4. Web–App Sync Plan — Manager Dashboard

Once the mobile app is complete, the web dashboard needs to consume the same backend. Here is the exact contract between the app and the web — what data the app produces and what the web needs to display.

### 4.1 Data Contract: What App Produces → What Web Consumes

| App Action                     | Backend Event                                                  | Web Dashboard Reaction                                                                           |
|--------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Employee taps Start Trip       | POST /trips/start → Socket.IO trip:started emitted             | New employee dot appears on Leaflet live map. Employee moves to 'Active' column in sidebar list. |
| GPS point recorded (every 15s) | Socket.IO location:update → location:broadcast to manager room | Employee dot moves on map. Speed/distance tooltip updates on hover.                              |
| Employee taps Pause            | PATCH /trips/:id/pause → Socket.IO trip:paused emitted         | Employee dot changes color to amber/yellow. 'Paused' badge shown next to name.                   |
| Employee taps Resume           | PATCH /trips/:id/resume → Socket.IO trip:resumed emitted       | Dot back to active color. Badge cleared.                                                         |
| Employee taps End Trip         | POST /trips/:id/end → Socket.IO trip:ended emitted             | Employee moves to 'Completed' column. Dot greyed out. New row appears in Claims queue.           |
| Employee generates claim       | POST /claims/generate/:trip_id → push to manager               | Claims approval table gets a new row. Manager sees notification badge increment.                 |
| Manager approves claim         | PATCH /claims/:id/approve → push to employee                   | Claim row moves from Pending to Approved. Employee sees push notification on phone.              |
| Manager rejects claim          | PATCH /claims/:id/reject → push to employee                    | Claim row moves to Rejected with reason. Employee sees push notification.                        |

### 4.2 Socket.IO Events the Web Must Listen To

| Event Name         | Payload                                                       | Web Handler Action                                                                        |
|--------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| location:broadcast | { employee_id, lat, lng, speed_kmh, trip_id, timestamp }      | Update Leaflet marker position for employee_id. Store last position in liveTrackingStore. |
| trip:started       | { trip_id, employee_id, employee_name, start_lat, start_lng } | Add new marker to map. Add to activeEmployees list.                                       |
| trip:ended         | { trip_id, employee_id, total_distance_km, duration_mins }    | Remove/grey marker. Update employee status. Refresh claims table.                         |
| trip:paused        | { trip_id, employee_id }                                      | Change marker color to amber. Show pause icon.                                            |
| trip:resumed       | { trip_id, employee_id }                                      | Change marker back to active blue.                                                        |

| Event Name       | Payload                                    | Web Handler Action                                                           |
|------------------|--------------------------------------------|------------------------------------------------------------------------------|
| claim:submitted  | { claim_id, employee_id, amount, trip_id } | Increment pending claims badge. Add row to claims table. Toast notification. |
| anomaly:detected | { trip_id, employee_id, type, detail }     | Show red alert banner on employee's map marker. Log in anomalies sidebar.    |

### 4.3 REST APIs the Web Dashboard Needs

| Endpoint                                      | Who Calls It                | When                                                                                                   |
|-----------------------------------------------|-----------------------------|--------------------------------------------------------------------------------------------------------|
| GET /api/trips/live                           | Manager on page load        | Fetch all currently active trips to populate live map on initial load (before Socket.IO events arrive) |
| GET /api/trips?employee_id=X&status=completed | Manager on employee profile | View full trip history for a specific employee                                                         |
| GET /api/trips/:id/locations                  | Manager on trip detail      | Fetch all GPS points to draw the full route polyline on route replay map                               |
| GET /api/claims?status=pending&manager_id=X   | Manager on claims page load | Fetch all pending claims from employees assigned to this manager                                       |
| PATCH /api/claims/:id/approve                 | Manager on claim approval   | Approve with optional adjusted amount                                                                  |
| PATCH /api/claims/:id/reject                  | Manager on claim rejection  | Reject with mandatory reason string                                                                    |
| GET /api/claims/export?format=csv             | Accountant                  | Download all approved claims as CSV for payroll                                                        |
| GET /api/users?role=employee&manager_id=X     | Manager on dashboard load   | Get list of assigned employees to show in sidebar                                                      |
| GET /api/analytics/summary?manager_id=X       | Manager dashboard           | Total distance, total claims, total approved amount this month for the manager's team                  |

### 4.4 Web Dashboard Pages to Build

| Page          | Route              | Primary Components                                                                                                             |
|---------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Live Map      | /manager/dashboard | Full-width Leaflet map, employee sidebar with online/offline/paused status, click employee to see details panel                |
| Claims Queue  | /manager/claims    | Table: employee name, trip date, distance, amount, status badge, Approve/Reject action buttons, filter by status/date/employee |
| Trip Detail   | /manager/trips/:id | Route replay map with polyline + start/end markers, stats grid, timeline, fraud flags if any                                   |
| Employee List | /manager/employees | Card/table of assigned employees, today's trip count, monthly distance, pending claims count per employee                      |
| Admin — Users | /admin/users       | Create user form, users table with role badges, activate/deactivate toggle, assign to manager dropdown                         |

| Page            | Route              | Primary Components                                                          |
|-----------------|--------------------|-----------------------------------------------------------------------------|
| Accountant View | /accountant/claims | Read-only approved claims table with CSV download button, month/date filter |

## 4.5 Web Dashboard Prompt — Give This to Your Web Dev



### Copy this prompt when you start web development:

I am building a Next.js web dashboard for a field employee GPS tracking system. The mobile app (React Native) is already built and the backend (Node.js + Socket.IO + PostgreSQL) is working.

Backend base URL: `https://your-api.onrender.com/api`

Socket.IO URL: `https://your-api.onrender.com (namespace: /tracking)`

**BUILD THIS FIRST (Manager Dashboard — Page 1):**

A live tracking page with:

1. Full-width Leaflet map (use react-leaflet, free, no API key needed)
2. On page load: call GET `/api/trips/live` → place a blue marker for each active employee
3. Connect to Socket.IO `/tracking` namespace with JWT token in auth header
4. Listen for 'location:broadcast' → update the relevant employee's marker position
5. Listen for 'trip:started' → add new marker
6. Listen for 'trip:ended' → grey out marker, show completed badge
7. Left sidebar: list of employees (online/paused/offline) with click to focus map on them
8. Click any marker → show popup: employee name, current speed, distance today, trip status

Tech: Next.js App Router, Tailwind CSS, Zustand for live tracking state, react-leaflet for map.

Use dynamic import for the Leaflet map (it requires 'use client' + no SSR).

Auth: JWT stored in memory (from login), sent as Authorization: Bearer token header.

## 5. Exact Development Order — What to Build Next

Follow this order strictly. Do not jump ahead. Each step has a clear 'done' condition.

### Phase 1: Fix Backend (Week 1) — Do First

| Day   | Task                                                                                  | Done When                                                                     |
|-------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Day 1 | Run Prompt 1 — fix distance calculation. Run the fix on existing trips.               | GET /trips/:id returns total_distance_km > 0 for a real completed trip        |
| Day 2 | Run Prompt 2 — complete trip lifecycle endpoints (start/pause/resume/end)             | Postman test: full trip lifecycle returns correct status transitions          |
| Day 3 | Run Prompt 3 — complete Socket.IO location handler with batch insert + Redis          | Emit 10 test location:update events → 10 rows appear in locations table       |
| Day 4 | Run Prompt 4 — complete claims API with generate + approve + reject                   | POST /claims/generate/:trip_id returns claim with correct INR amount          |
| Day 5 | Run Prompt 5 — DB audit. Fix any data issues found. Run Prompt 6 — push notifications | All DB audit queries return 0 rows. Push notification arrives on test device. |

### Phase 2: Complete App UI (Week 2)

| Day    | Task                                                                               | Done When                                                                         |
|--------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Day 6  | Build Active Trip Screen with live timer, distance, Pause/Resume/End buttons       | Full trip can be started, paused, resumed, ended from the screen                  |
| Day 7  | Connect location tracking to Active Trip Screen — emit Socket.IO on each GPS point | Manager can see employee dot move on web during test trip                         |
| Day 8  | Add Generate Claim button to Trip Details screen                                   | Completed trip shows button → tapping creates claim → Claims screen shows pending |
| Day 9  | Fix map coordinates (remove hardcoded Seattle) + fix 0.00 km display               | Trip Details shows actual route and correct distance                              |
| Day 10 | Add push notification handler on mobile (navigate to claim on tap)                 | Tapping push notification opens the correct claim screen                          |

### Phase 3: App Polish (Week 3)

| Task                                                                    | Priority | Done When                                                                     |
|-------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------|
| Offline queue for GPS points (AsyncStorage buffer + flush on reconnect) | High     | Trip recorded correctly after going through a tunnel (no signal)              |
| Empty states for Trips and Claims screens                               | Medium   | New employee account shows helpful empty state, not blank screen              |
| Confirmation modal before End Trip                                      | High     | Accidental end trip impossible without confirming distance + estimated amount |
| Replace UUID with friendly employee code on Profile                     | Low      | Profile shows EMP-XXXXXX format, not raw UUID                                 |

| Task                           | Priority | Done When                                                  |
|--------------------------------|----------|------------------------------------------------------------|
| Fix Region: Unknown on Profile | Low      | Region shows actual city/state from device or org_settings |

## Phase 4: Web Dashboard (After App is Complete)



### Start web only after you confirm these from the app:

- ☒ Full trip lifecycle works end-to-end (start → track → end → claim generated)
  - ☒ Distance is calculating correctly
  - ☒ Socket.IO location:broadcast is being emitted from backend
  - ☒ At least one real test trip exists in the database with > 5 location points
  - ☒ Claim approve/reject API is tested via Postman
- When all 5 are true, use the web dashboard prompt in Section 4.5 to start the web.

## 6. Testing Checklist — Verify Everything Works

Run these manual tests in order after completing each phase. Use 2 devices or 2 browsers (one as employee, one as manager).

### Backend API Tests (Postman or curl)

| Test            | Command                                 | Expected Result                              |
|-----------------|-----------------------------------------|----------------------------------------------|
| Login           | POST /auth/login {email, password}      | Returns accessToken + user.role              |
| Start trip      | POST /trips/start with Bearer token     | Returns { trip_id, status: 'active' }        |
| Send location   | Socket.IO emit location:update          | Row appears in locations table immediately   |
| End trip        | POST /trips/:id/end                     | Returns { total_distance_km: > 0, claim_id } |
| Generate claim  | POST /claims/generate/:trip_id          | Returns { claim_id, calculated_amount: > 0 } |
| Manager approve | PATCH /claims/:id/approve (manager JWT) | Claim status = 'approved' in DB              |
| Live positions  | GET /trips/live (manager JWT)           | Returns array of active trips with lat/lng   |

### End-to-End Flow Test (2 Devices)

| Step | Device 1 (Employee Phone)            | Device 2 (Manager Web/Browser)   | Pass Condition                                                     |
|------|--------------------------------------|----------------------------------|--------------------------------------------------------------------|
| 1    | Login as employee                    | Login as manager → open Live Map | Both logged in, map shows no active trips                          |
| 2    | Tap Start Trip on Dashboard          | —                                | API returns trip_id, employee navigates to Active Trip screen      |
| 3    | Walk around for 2 minutes (real GPS) | Watch map                        | Employee dot appears on manager's map and moves as employee walks  |
| 4    | Tap Pause                            | Watch map                        | Dot changes color/icon to 'paused' on manager map                  |
| 5    | Tap Resume                           | Watch map                        | Dot back to active                                                 |
| 6    | Tap End Trip → confirm               | Watch map                        | Dot greys out, employee sees distance > 0 km                       |
| 7    | Tap Generate Claim on Trip Details   | Watch Claims table               | New pending claim row appears in manager's claim table             |
| 8    | Manager approves claim               | Tap Approve on claim             | Employee phone receives push notification 'Claim Approved — INR X' |
| 9    | Employee checks Claims tab           | —                                | Claim shows 'Approved' status badge with correct amount            |